

Universidad Politécnica de San Luis Potosí

**LAB: FILE PATH TRAVERSAL SEQUENCES STRIPPED WITH
SUPERFLUOUS URL-DECODE**

Presentado por:

Christian Alfredo Morales Meza - 184937

Asignatura: Fundamentos de Ciberseguridad

Profesor: Servando López Contreras

Fecha de entrega: 09/09/2026

Índice de contenido

Índice de contenido	2
Ficha del laboratorio	3
Objetivo del laboratorio	3
Explicación técnica de la vulnerabilidad	3
Procedimiento paso a paso.....	3
Evidencia del procedimiento.....	4
Evidencias obligatorias	9
Explicación del payload.....	11
Mitigación recomendada.....	12

Ficha del laboratorio

Laboratorio: LAB: FILE PATH TRAVERSAL SEQUENCES STRIPPED WITH SUPERFLUOUS URL-DECODE

Nivel: Practitioner

Tipo de explotación: File Path Traversal mediante evasión de desinfección con doble codificación URL

Objetivo del laboratorio

Acceder y exfiltrar el contenido del archivo de configuración del sistema operativo `/etc/passwd` evadiendo los filtros de seguridad de la aplicación web.

Explicación técnica de la vulnerabilidad

La causa raíz de esta vulnerabilidad reside en una discrepancia arquitectónica entre la etapa de validación de entradas de la aplicación y la posterior resolución del sistema de archivos en el backend. El servidor web implementa un filtro de desinfección de entrada diseñado para buscar y bloquear secuencias de salto de directorio estándar (como `../`). Sin embargo, este filtro actúa en una fase en la que los datos del parámetro de entrada aún conservan un nivel de codificación URL. Al procesar una entrada doblemente codificada, los caracteres maliciosos (por ejemplo, los puntos y las barras) se presentan en formatos no literales (como `%252e` o `%252f`), lo que permite que la lógica del filtro apruebe la petición al no identificar coincidencias en su firma o lista negra. Posteriormente, el motor de la aplicación o la API del sistema de archivos ejecuta un proceso de decodificación de URL redundante ('superfluo') antes de interactuar directamente con la API del sistema operativo. Esto reconstruye la secuencia destructiva de escape de directorios en tiempo de ejecución, rompiendo los límites del aislamiento de archivos e impactando directamente la Confidencialidad del servidor, bajo la categoría A01:2021-Broken Access Control del OWASP Top 10.

Procedimiento paso a paso

Iniciamos el entorno de auditoría encendiendo la función de intercepción de peticiones en la pestaña Proxy de la herramienta Burp Suite. Esto nos permitirá capturar y manipular el tráfico generado por el navegador embebido.

Accedemos al portal web del laboratorio, el cual consiste en una tienda en línea funcional. Navegamos por el sitio web para cargar imágenes de diversos productos y así poblar el historial de tráfico HTTP con solicitudes dinámicas.

Analizamos el historial de peticiones HTTP en la pestaña 'HTTP history' de Burp Suite, enfocando nuestra atención en las llamadas que interactúan directamente con el sistema de archivos del servidor, específicamente el endpoint `/image?filename=`.

Seleccionamos una petición GET válida orientada a la carga de una imagen de producto (como `'64.jpg'`) y la transferimos al módulo Repeater presionando la combinación de teclas `Ctrl + R`, lo que nos facultará para realizar pruebas iterativas.

Dentro de Repeater, enviamos la solicitud sin modificaciones para verificar la respuesta del servidor; confirmamos que el backend retorna un código HTTP 200 OK junto con el flujo binario correspondiente a la imagen solicitada.

Modificamos el parámetro 'filename' introduciendo el payload convencional de cruce de directorios '.././.././etc/passwd'. Al enviar la petición, el servidor responde con un código de estado 400 Bad Request que contiene el mensaje 'No such file', evidenciando que el filtro de seguridad bloquea secuencias explícitas.

Procedemos a evadir el filtro codificando el payload. Seleccionamos la cadena inyectada en Repeater, abrimos el menú contextual con un clic derecho, y navegamos a 'Convert selection' -> 'URL' -> 'URL-encode all characters' para aplicar una codificación de primer nivel.

Transmitimos la solicitud con codificación URL simple. Sin embargo, el servidor web decodifica automáticamente la petición una vez antes de pasarla al filtro, lo que ocasiona que este vuelva a detectar y neutralizar la secuencia maliciosa, devolviendo nuevamente un error HTTP 400.

Para superar esta validación, aplicamos un segundo nivel de codificación URL completa sobre toda la secuencia ya codificada en el editor de peticiones de Burp Repeater, convirtiendo cada carácter especial de porcentaje en su representación hexadecimal correspondiente.

Enviamos la solicitud con el payload doblemente codificado. El servidor frontal realiza la primera decodificación, el filtro analiza el contenido intermedio considerándolo seguro, y finalmente la decodificación 'superflua' en el backend restaura la secuencia real, sirviendo con éxito el archivo '/etc/passwd' con código HTTP 200 OK.

Evidencia del procedimiento

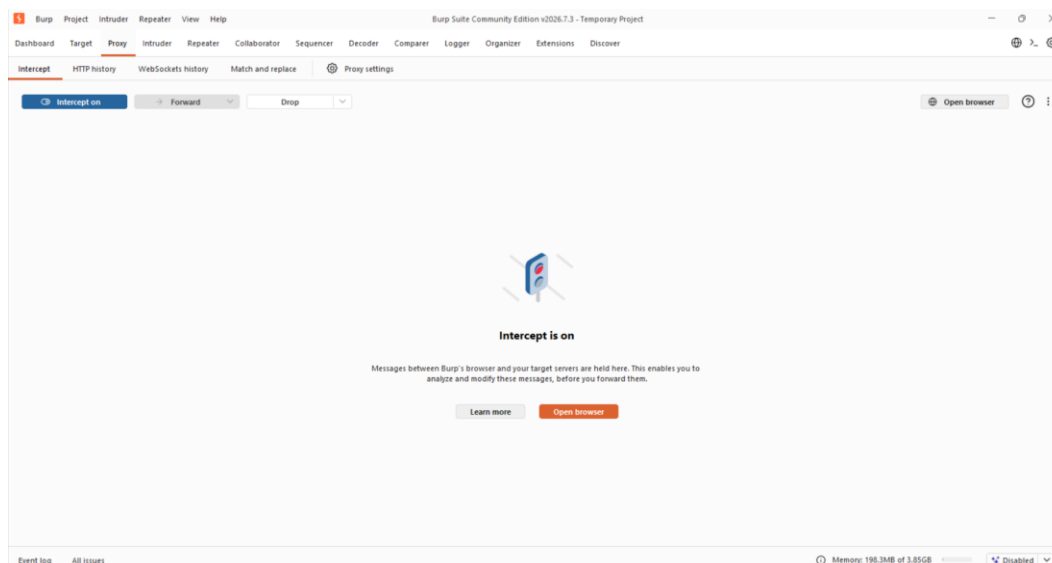


Figura 1. Activación del proxy de intercepción en Burp Suite para capturar y analizar el tráfico de red del laboratorio.

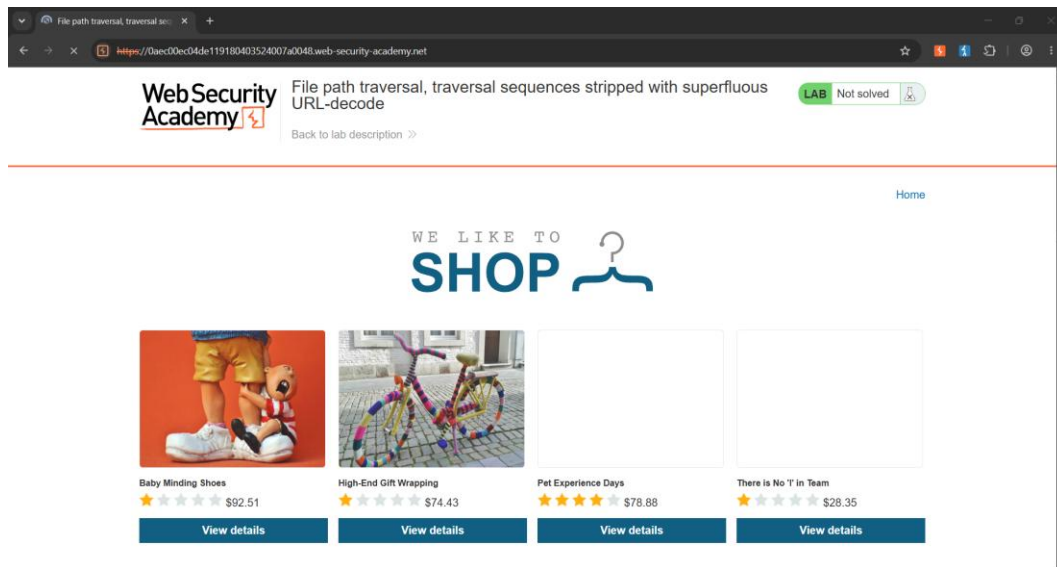


Figura 2. Navegación en el sitio web de la tienda de comercio electrónico para generar solicitudes dirigidas al servidor web.

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

⌕

Figura 3. Identificación de peticiones GET vulnerables en el historial HTTP asociadas con la carga de imágenes de productos.

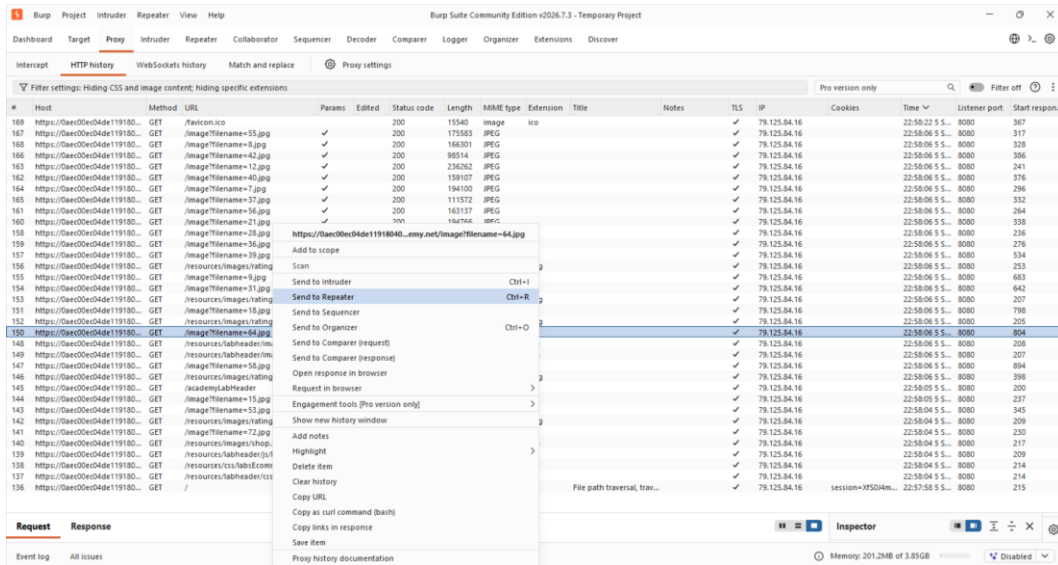


Figura 4. Envío de una solicitud de imagen seleccionada al módulo Repeater de Burp Suite para su modificación controlada.

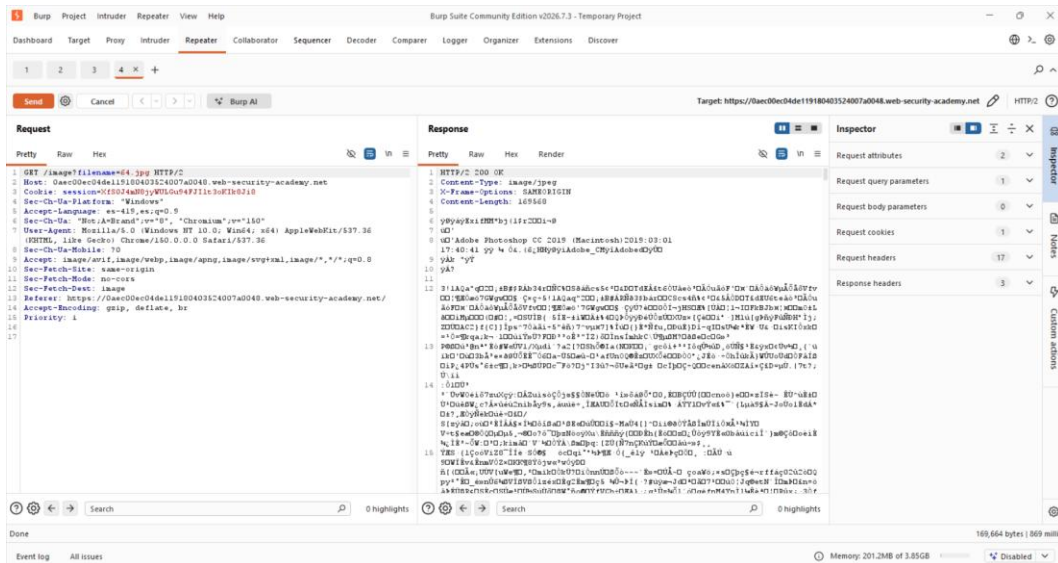


Figura 5. Validación de la respuesta original del servidor mediante el envío de la petición sin alteraciones en Repeater.

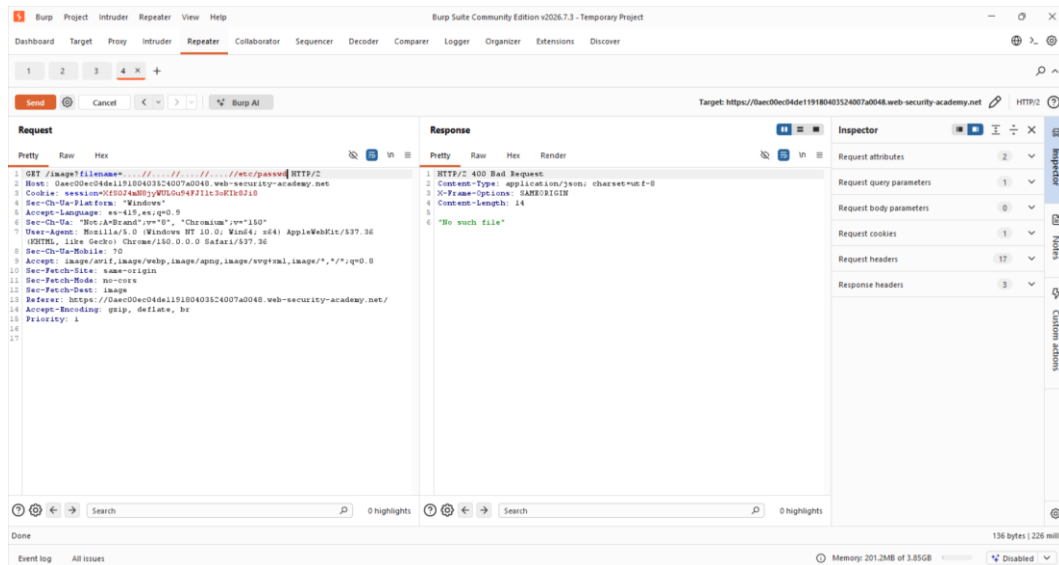


Figura 6. Prueba de inyección de un payload clásico de salto de directorio que es rechazado con un error 400 Bad Request.

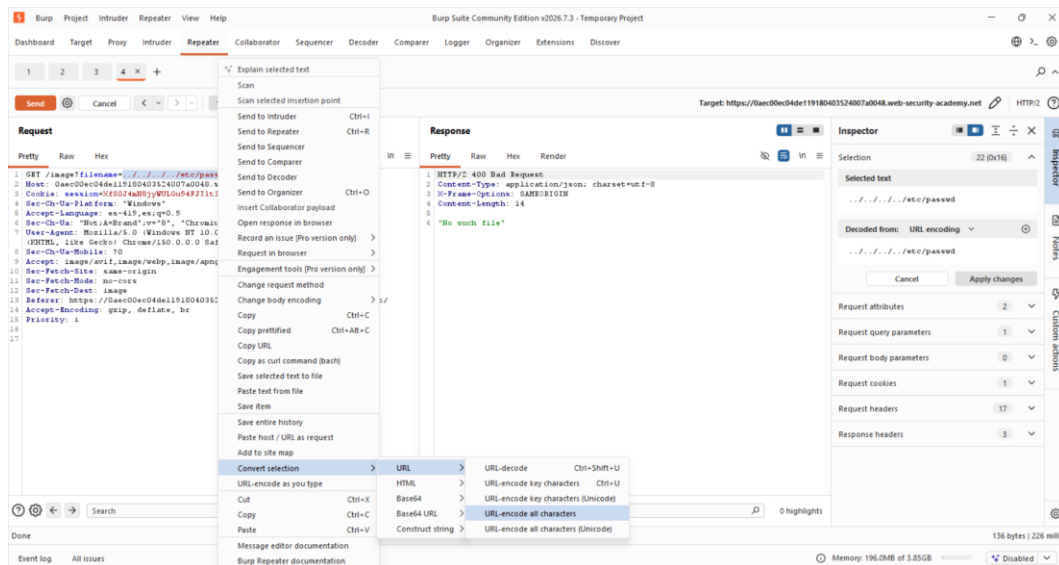


Figura 7. Aplicación de la opción de codificación URL completa en Burp Suite para transformar la cadena del payload.

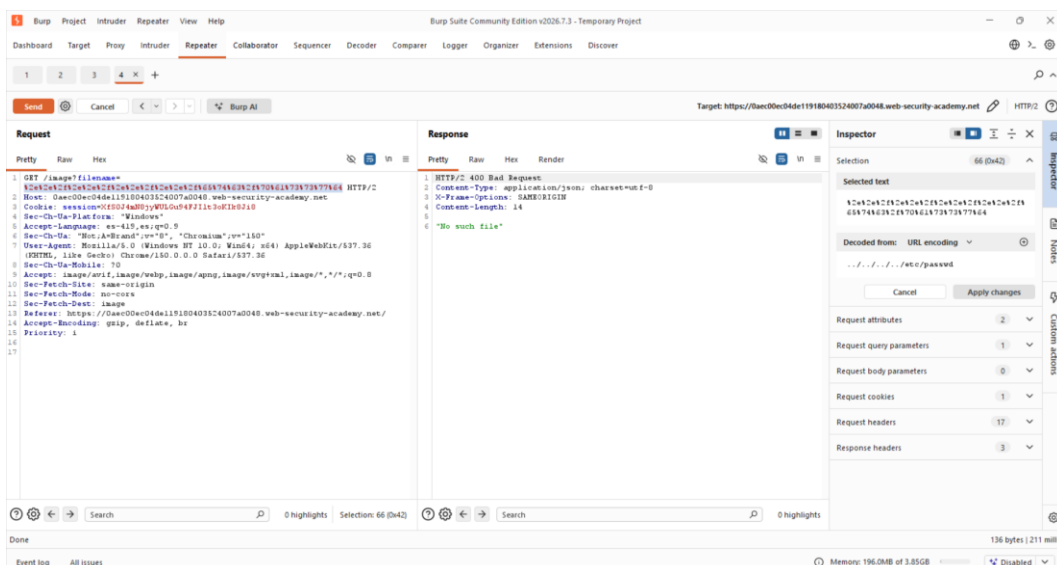


Figura 8. Envío de la petición con codificación URL simple que resulta bloqueada debido al filtrado reactivo del backend.

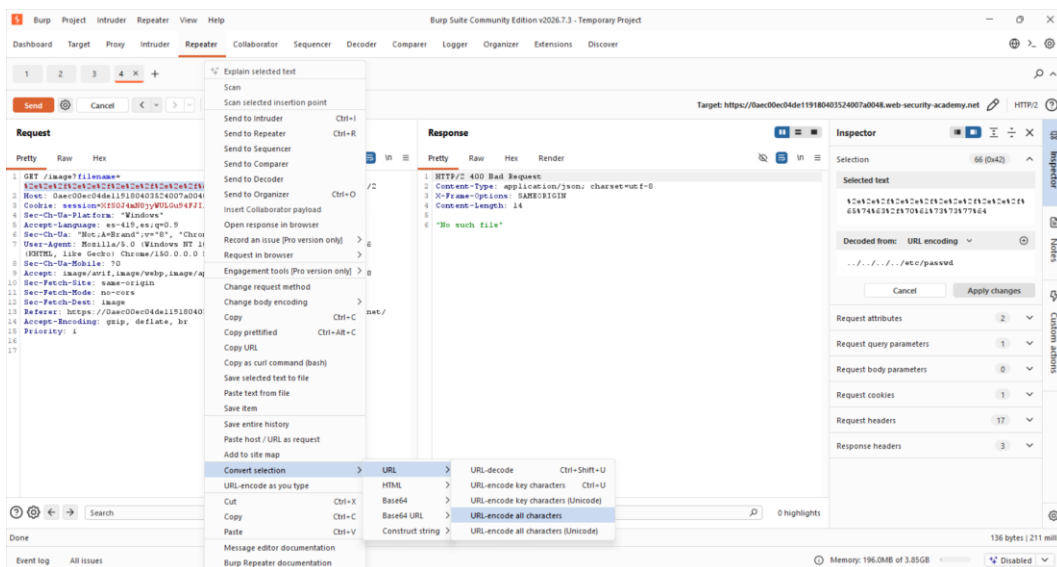


Figura 9. Generación de la doble codificación URL sobre el payload utilizando nuevamente las herramientas de conversión de Burp.

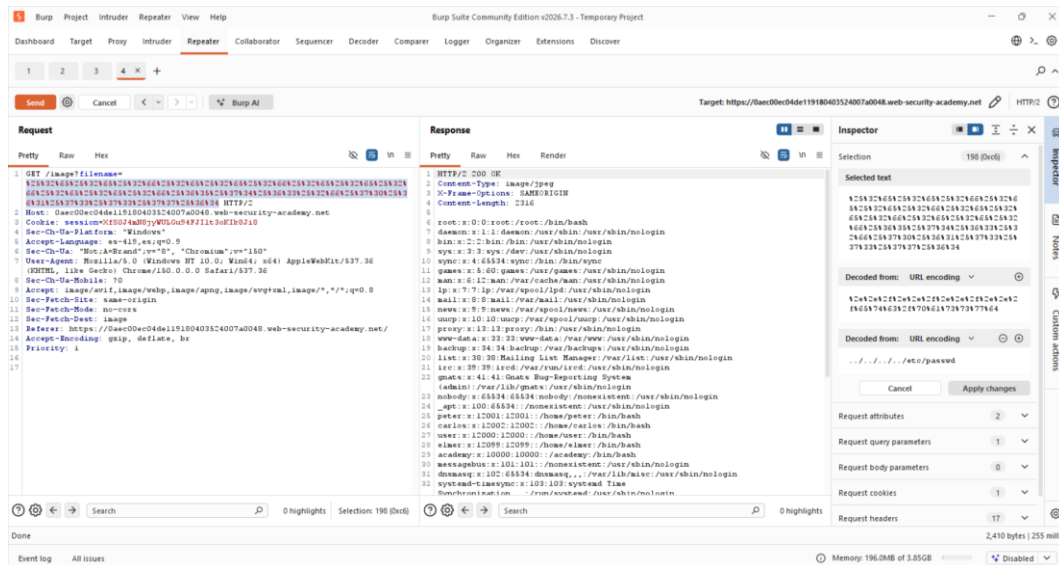


Figura 10. Ejecución exitosa del ataque tras evadir el filtro con doble codificación, visualizando el contenido de /etc/passwd.

Evidencias obligatorias

Request interceptado

Burp

Project

Intruder

Repeater

View

Help

Burp Suite Community Edition V2023.7.4 - Temporary Project

Dashboard

Target

Proxy

Intruder

Repeater

Collaborator

Sequencer

Decoder

Comparer

Logger

Organizer

Extensions

Discover

Intercept

HTTP history

WebSockets history

Match and replace

Proxy settings

Filter settings: Hiding CSS and image content; hiding specific extensions

Pre version only

Filter off

#	Host	Method	URL	Params	Edited	Status code	Length	MIME type	Extension	Title	Notes	TLS	IP	Cookies	Time vs	Lintner port	Start position
169	https://baec0e0b4de119180...	GET	/resources/logo			200	15940	image	ico			✓	79.125.94.16		22:58:22 S...	8000	367
167	https://baec0e0b4de119180...	GET	/image?filename=5.jpg		✓	200	175583	JPEG				✓	79.125.94.16		22:58:06 S...	8000	317
168	https://baec0e0b4de119180...	GET	/image?filename=8.jpg		✓	200	166301	JPEG				✓	79.125.94.16		22:58:05 S...	8000	328
166	https://baec0e0b4de119180...	GET	/image?filename=42.jpg		✓	200	90514	JPEG				✓	79.125.94.16		22:58:06 S...	8000	386
163	https://baec0e0b4de119180...	GET	/image?filename=12.jpg		✓	200	236282	JPEG				✓	79.125.94.16		22:58:06 S...	8000	241
162	https://baec0e0b4de119180...	GET	/image?filename=40.jpg		✓	200	159107	JPEG				✓	79.125.94.16		22:58:06 S...	8000	376
164	https://baec0e0b4de119180...	GET	/image?filename=7.jpg		✓	200	194400	JPEG				✓	79.125.94.16		22:58:06 S...	8000	296
161	https://baec0e0b4de119180...	GET	/image?filename=37.jpg		✓	200	133514	JPEG				✓	79.125.94.16		22:58:06 S...	8000	332
165	https://baec0e0b4de119180...	GET	/image?filename=56.jpg		✓	200	163137	JPEG				✓	79.125.94.16		22:58:06 S...	8000	264
160	https://baec0e0b4de119180...	GET	/image?filename=21.jpg		✓	200	194766	JPEG				✓	79.125.94.16		22:58:06 S...	8000	338
157	https://baec0e0b4de119180...	GET	/image?filename=28.jpg		✓	200	127703	JPEG				✓	79.125.94.16		22:58:06 S...	8000	236
159	https://baec0e0b4de119180...	GET	/image?filename=36.jpg		✓	200	152678	JPEG				✓	79.125.94.16		22:58:06 S...	8000	276
157	https://baec0e0b4de119180...	GET	/image?filename=39.jpg		✓	200	156821	JPEG				✓	79.125.94.16		22:58:06 S...	8000	534
156	https://baec0e0b4de119180...	GET	/resources/images/hotint6.png		✓	200	740	PNG	png			✓	79.125.94.16		22:58:06 S...	8000	253
155	https://baec0e0b4de119180...	GET	/image?filename=8.jpg		✓	200	166611	JPEG				✓	79.125.94.16		22:58:06 S...	8000	241
154	https://baec0e0b4de119180...	GET	/image?filename=31.jpg		✓	200	127867	JPEG				✓	79.125.94.16		22:58:06 S...	8000	642
153	https://baec0e0b4de119180...	GET	/resources/images/hotint5.png		✓	200	624	PNG	png			✓	79.125.94.16		22:58:06 S...	8000	207
152	https://baec0e0b4de119180...	GET	/resources/images/logo		✓	200	133514	JPEG				✓	79.125.94.16		22:58:06 S...	8000	798
152	https://baec0e0b4de119180...	GET	/resources/images/hotint3.png		✓	200	758	PNG	png			✓	79.125.94.16		22:58:06 S...	8000	205
150	https://baec0e0b4de119180...	GET	/image?filename=64.jpg		✓	200	169964	JPEG				✓	79.125.94.16		22:58:06 S...	8000	804
148	https://baec0e0b4de119180...	GET	/resources/labels/header/logo.jpg...		✓	200	8523	SVG	svg			✓	79.125.94.16		22:58:06 S...	8000	200
149	https://baec0e0b4de119180...	GET	/resources/labels/header/logo.jpg...		✓	200	942	SVG	svg			✓	79.125.94.16		22:58:06 S...	8000	207
147	https://baec0e0b4de119180...	GET	/image?filename=58.jpg		✓	200	163688	JPEG				✓	79.125.94.16		22:58:06 S...	8000	894
146	https://baec0e0b4de119180...	GET	/resources/images/hotint4.png		✓	200	742	PNG	png			✓	79.125.94.16		22:58:06 S...	8000	398
145	https://baec0e0b4de119180...	GET	/resources/labels/header		✓	200	430	JSON				✓	79.125.94.16		22:58:06 S...	8000	209
144	https://baec0e0b4de119180...	GET	/image?filename=15.jpg		✓	200	131445	JPEG				✓	79.125.94.16		22:58:05 S...	8000	237
143	https://baec0e0b4de119180...	GET	/image?filename=53.jpg		✓	200	201401	JPEG				✓	79.125.94.16		22:58:06 S...	8000	345
142	https://baec0e0b4de119180...	GET	/resources/images/hotint1.png		✓	200	708	PNG	png			✓	79.125.94.16		22:58:06 S...	8000	209
141	https://baec0e0b4de119180...	GET	/image?filename=72.jpg		✓	200	212227	JPEG				✓	79.125.94.16		22:58:06 S...	8000	230
140	https://baec0e0b4de119180...	GET	/resources/images/hotip.svg		✓	200	7258	SVG	svg			✓	79.125.94.16		22:58:06 S...	8000	217
139	https://baec0e0b4de119180...	GET	/resources/labels/header/logo.jpg		✓	200	1873	image/jpeg				✓	79.125.94.16		22:58:06 S...	8000	209
138	https://baec0e0b4de119180...	GET	/resources/css/labels/commerce.css		✓	200	20347	CSS	css			✓	79.125.94.16		22:58:06 S...	8000	214
137	https://baec0e0b4de119180...	GET	/resources/labels/css/academy...		✓	200	5771	CSS	css			✓	79.125.94.16		22:58:06 S...	8000	214
136	https://baec0e0b4de119180...	GET	/		✓	200	10641	HTML		File path traversal, tra...		✓	79.125.94.16	session=YF5dM...	22:57:58 S...	8000	215

Request

Response

Inspector

Event log

All issues

Memory: 196.0MB of 3.1GB

Disabled

Figura 11. Request interceptado — Petición HTTP original seleccionada en el historial de navegación para cargar el recurso de imagen mediante el parámetro vulnerable 'filename'.

Payload utilizado



Figura 12. Payload utilizado — Detalle del payload modificado y doblemente codificado en URL dentro del panel de edición de solicitudes de Burp Repeater.

Respuesta vulnerable

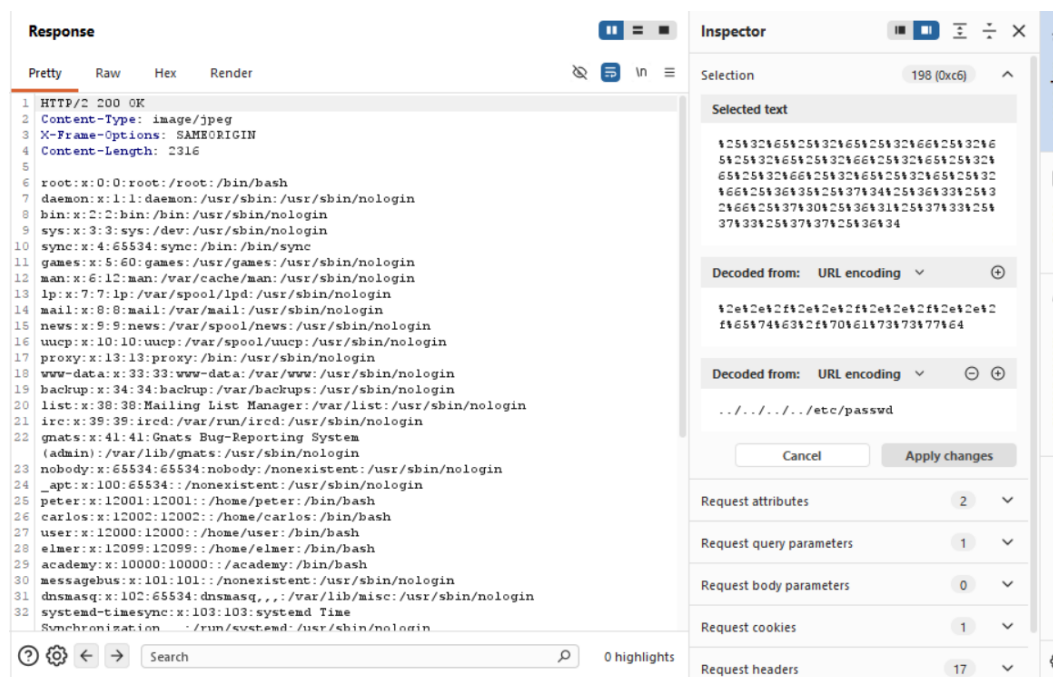


Figura 13. Respuesta vulnerable — Respuesta HTTP 200 OK del servidor vulnerable que expone en texto claro el contenido completo del archivo `'/etc/passwd'`.

Confirmación de “Lab Solved”

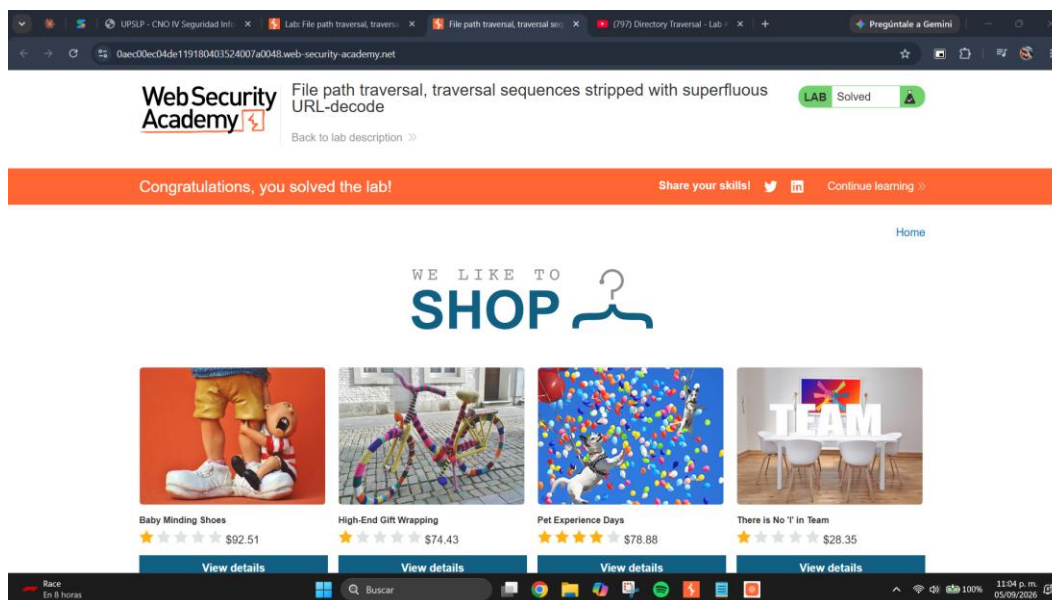


Figura 14. Confirmación de “Lab Solved” — Confirmación en pantalla del laboratorio de PortSwigger con el banner que valida la resolución exitosa del desafío de ciberseguridad.

Explicación del payload

El payload final empleado en la explotación es la versión doblemente codificada en URL del clásico vector de escape de directorios: `'../../../../../etc/passwd'`. En una codificación URL convencional, los caracteres de control `'.'` y `'/'` se transforman en `'%2e'` y `'%2f'` respectivamente. Al codificar estos valores por segunda vez de manera exhaustiva ('URL-encode all characters' en Burp Suite), el signo de porcentaje `'%'` se convierte en su valor hex `'%25'` y los caracteres alfanuméricos restantes de la codificación se traducen individualmente. Así, la secuencia de control se procesa como `'%25%33%32%25%36%35%25%33%32%25%36%35%25%32%33...'` en su forma completamente codificada por Burp, o de forma simplificada para los separadores como `'%252e%252e%252f'`.

Cuando el servidor web intercepta esta entrada, ejecuta la decodificación por defecto de la capa de transporte HTTP, convirtiendo el payload a `'%2e%2e%2f%2e%2e%2f%2e%2e%2f%2e%2e%2fetc%2fpasswd'`. El filtro de la aplicación inspecciona esta cadena y, al no encontrar los caracteres literales `'../'`, asume que la entrada es inocua y permite su avance. Posteriormente, antes de realizar la llamada a las funciones de lectura de archivos, el backend ejecuta un segundo proceso de decodificación de URL (superfluo), restaurando los caracteres funcionales de navegación que permiten al sistema de archivos del sistema operativo subir en la jerarquía hasta la raíz y acceder de forma ilegítima al archivo de usuarios del sistema.

Mitigación recomendada

Para mitigar eficazmente este riesgo técnico, se debe aplicar una estrategia de defensa en profundidad basada en los siguientes controles:

1. **Remediación a nivel de código (Canonización y Validación):** Evitar por completo pasar entradas proporcionadas por el usuario de forma directa a las APIs de archivos. Si es estrictamente necesario, se debe implementar una validación rigurosa basada en listas blancas (whitelists) que solo acepte caracteres alfanuméricos simples. Inmediatamente después de recibir la entrada, se debe resolver la ruta absoluta utilizando mecanismos de canonización nativos del lenguaje (por ejemplo, 'getCanonicalPath()' en Java o 'realpath()' en PHP) y verificar explícitamente que la ruta resuelta comience estrictamente con el directorio base predefinido (ej. /var/www/images/).
2. **Configuración de Servidores:** Deshabilitar o unificar las políticas de decodificación URL redundantes en el ecosistema (servidor proxy inverso, servidor web y motor de base de datos) para evitar que se procesen entradas doblemente codificadas de manera inconsistente.
3. **Principio de menor privilegio:** El servidor web y el motor de la aplicación deben ejecutarse bajo cuentas de sistema operativo restrictivas sin acceso interactivo de shell (como 'www-data') y con permisos estrictos de lectura limitados exclusivamente a su propio directorio raíz de trabajo.